Stack lines

September
2025


# Requirements Documentation ↗

# Introduction

## 1.1 Purpose of the Document

Documentation of Requirements (RD) describes the requirements for Jaguard to operate. Jaguard provides breach monitoring solutions via the cloud. In this document, developers and project managers can review all requirements and agree on their implementation before the project begins. The document was created based on the market analysis and review of current breach monitoring systems.

## 1.2 Major Problems and Project Goals

In today's world, data breaches are becoming increasingly common and has significant risks to individuals as well as organizations. In many existing breach platforms, excessive personal information is collected or the protection is weak and the company is not transparent about their solution.

Our primary goal is to provide a reliable, secure, cloud-based monitoring platform that identifies potential data breaches while safeguarding user privacy. This project aims to deliver an easy-to-use, scalable, and reliable solution for individuals and organizations.

## 1.3 Proposed System Overview and Configuration Chart

Jaguard operates as a cloud-based application that continuously monitors and analyzes data sources to detect compromised credentials and security breaches using multiple APIs. Users can monitor multiple emails for data breaches. To ensure maximum privacy, the system uses an encrypted algorithm that avoids storing personal information.

# Introduction (continued)

## 1.3 Proposed System Overview and Configuration Chart

Jaguard integrates multiple APIs to detect the exposed data and displays it as visual reports and breach statistics through its dashboard. The key features of of the system include:

- **Login Module:** Secure account with password hash and two factor authentication.
- **AI Security Assistant:** A chatbot that provides personalized security recommendations.
- **Breach Reporting:** Reports from multiple sources showing whether the data has been compromised or not.

Bu design, our system provides security and privacy. Sensitive data is protected with encryption and secure session management to ensure that no personal information is stored beyond what is strictly necessary. With Jaguard, we combine modern security practices with scalable cloud infrastructure and AI-driven insights to provide everyday users with professional security tools.

Figure 1 on the following page is the System Configuration Chart for Jaguard.

# Introduction (continued)

## 1.3 Proposed System Overview and Configuration Chart

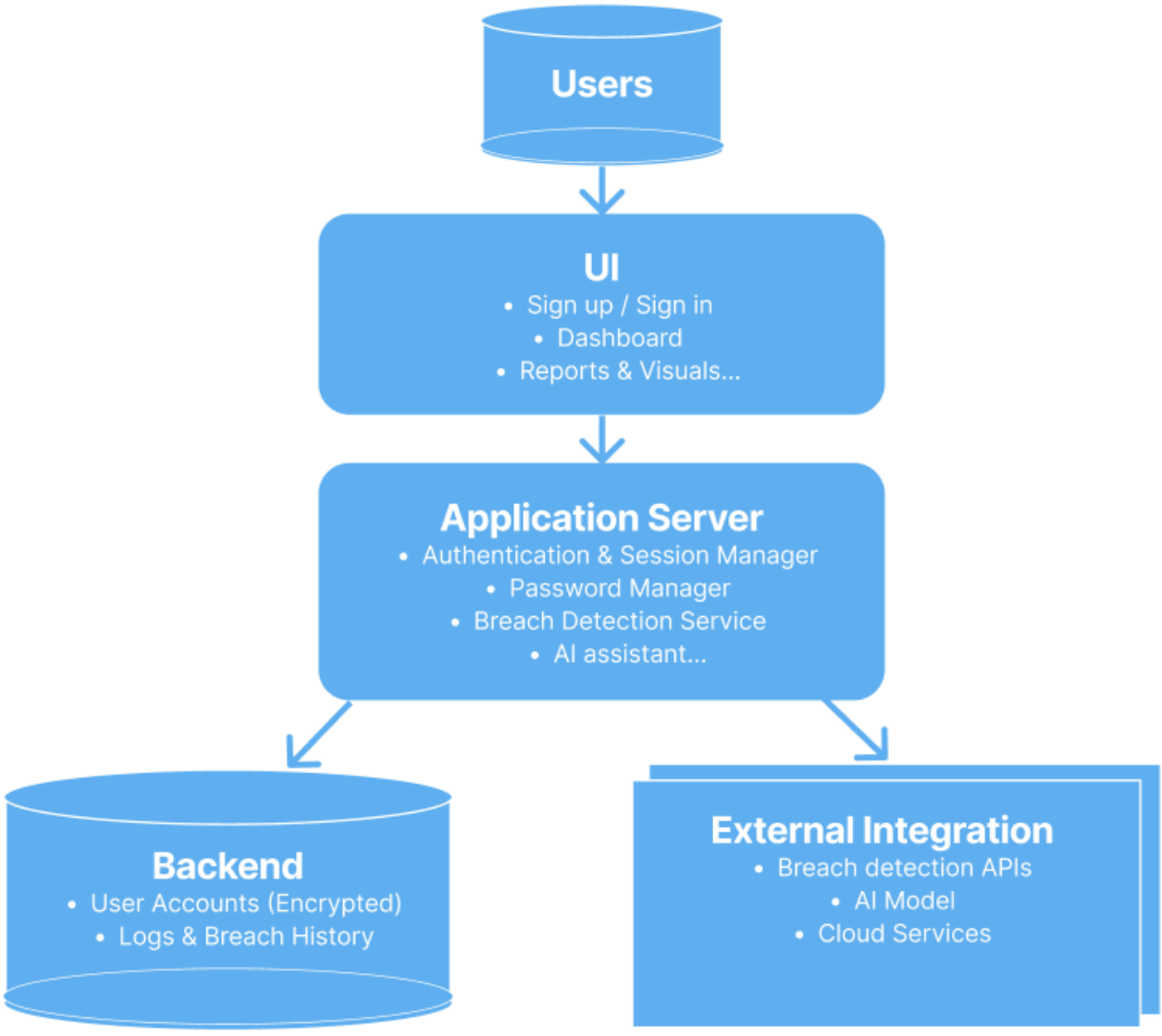



**Figure 1**

# Introduction (continued)

## 1.4 Definitions, Acronyms, and Abbreviations

| | |
|---|---|
| **2-FA** | **Two Factor Authentication** |
| **API** | **Application Programming Interface** |
| **Auth** | **Authentication** |
| **DB** | **Database** |
| **ER Diagram** | **Entity Relationship Diagram** |
| **JWT** | **Json Web Token** |
| **Jaguard** | **Name of the system - cloud based data breach search system** |
| **RD** | **Requirement Documentation** |
| **UI** | **User Interface** |
| **UML** | **Unified Modeling Language** |
| **WBS** | **Work Breakdown Structure** |
| **CR** | **Change Request** |

## Overview of Rest of RD

- **General Description** – Provides product perspective, user needs, system constraints, and assumptions.
- **Software Requirements** – Details both functional and non-functional requirements, including interfaces, workflows, and exception handling.
- **Log of Meetings, Reviews, and Change Control** – Records communication, decisions, and requirement updates.
- **Appendix** – Contains diagrams, charts, work breakdown structure, glossary of terms, and approval signatures.

# General Description

## 2.1 Product Perspective

- The data breach website is going to be a standalone product that user can utilize to check if their email has been compromised.
- Additionally, the website will be connected to APIs, which will be how the data for the breaches is obtained. Furthermore, it will also connect to a database which will only have the email address and password of the user.
- Some of the key features the website will be offering are the Login Module, Data Privacy, AI Model/Report prediction suggestions, and report generation, as well as Chatbot user Education, Data breach finder, and Visuals/ Statistics of breaches.
- Will have a chatbot to educate users or Visuals of breaches such as graphs of what days or months have more data breaches.
- With the reports the data breach information is concise and informative.

## 2.2 User Requirements

- Users can expect either to login to the website if they have created an account or they can utilize the guest option and search for specific email address.
- Additionally, users will be able to use the chatbot and ask it what actions users should take for a data breach.
- Also, users can have reports generated with a summary of the data breach for whatever email address they wanted.
- The website's performance will be high because users will be able to access each page quickly.
- The website is very intuitive and there are only a few buttons that the users must press to get their email address checked for data breaches which makes it easy for all users to use.

# General Description (cont)

## 2.2 User Requirements (cont)

- The key features that are critical are the data breach finder being able to find the data breaches.
- Additionally, report generation is key because users need to be able to see a report of what websites on their email address has been part of a data breach.
- Furthermore, the data privacy feature is important in keeping users' data safe and keeping it from bad actors.
- The features that are nice to have are the login module, Chatbot user education, and AI model/ report prediction, and the visuals/statistics.
- These features add quality of life to the website, allowing users to both navigate and understand their data breach easier.

## 2.3 User Characteristics

- The website will be used by technical staff, system administrators, software developers, and customers.
- The technical staff will be assisting the customers with any questions or concerns they may have about the website. They will need training to know how to use the website effectively and be knowledgeable enough about the software to help customers with issues.
- The system administrators will be handling the security of the website and making sure that the website is reliable and always available to users; they will also be working on the maintenance of the website such as making sure the website has up to date software.

# General Description (cont)

## 2.3 User Characteristics (cont)

- Some of the technical skills they need to have are experience in dealing with system security, and how to implement it effectively.
- Additionally, they need to know how to improve the website's performance, and they need to be able to maintain backup servers.
- The software developers will oversee the adding of features that technical staff or customers may want as well as fixing any broken features.
- Software developers are important because they fix features, as well as create new features that help make the website less stagnant.
- The customers are going to be using the website to check if their email address has been part of a data breach. The main technical skill users need to know is how to use a laptop or mobile device.
- Additionally, the website will have many different languages that a user can choose from to help the user better traverse the website.

## 2.4 General Constraints

- Some of the main constraints that our website has are the budget, time, software/hardware limitation, and ethical constraints.
- The main budget constraints are balancing expenses and profits when implementing the website.
- Firstly, one of the main expenses is paying our developers for building and maintaining the website.
- Another expense is paying for the software to develop the websites since we will be using the premium versions of these software tools to make the website responsive, reliable, and maintainable.
- Furthermore, the expense for paying for the premium versions of third-party APIs is another budget constraint.

# General Description (cont)

## 2.4 General Constraints (cont)

- Secondly, time is one of the most impactful constraints that we have because we must have our website fully documented, and all our features implemented within a three-month period.
- Additionally, the development team needs enough time to fully test the website's features and make sure that they are functional.
- Thirdly, the main software/hardware limitation that the team is facing is training our AI model to work on our website.
- Having more members to assign and assist with tasks would make the training of the AI faster and more efficient.
- The main ethical constraint that our website faces is data privacy; we are using SHA-256 hash to store users' passwords to prevent exposure of a plain text password.
- Additionally, the website will implement access control to be role based so that not all users of the website have the same permissions.
- Furthermore, we will have two factor authentication enabled when users are logging in to their account so that authorized users are accessing their account.

# General Description (cont)

## 2.5 Assumptions and Dependencies

- The assumptions that go into creating and using the website are user background, training, and 3rd party assets.
- The users will need to have access to a computer or a mobile device.
- Some assumptions that we have made about the user's background are that they are knowledgeable in using computers and mobile devices to type their email into the text box.
- Additionally, users should have reliable access to the internet, so that their session does not get disconnected when they are using the website.
- The main dependencies we need for the website will be third-party APIs that we will call to get the breach data that will be used for our reports, and statistics.
- Additionally, we will use Twilio to implement the Two-Factor authentication for the login module.
- Furthermore, MySQL Server is another third-party dependency that the website will be using. This is where our user's information will be stored in the database. The main data that's stored in the database is the email address and the user's SHA-256 hashed password.

# Traceability Matrix Functional Requirements

**Legend**

| Color | Meaning |
|---|---|
| | Completed |
| | In Progress |
| | Not Started |

| Req ID | Requirement Description | Type | Design Component | Implementation Module | Test Case | Status |
|---|---|---|---|---|---|---|
| **FR-01** | System shall provide a **Login User Interface** that will allow a user to enter their credentials. | Functional Requirement | Login Page, UI Workflow, External Interfaces, Exception handling, User Table Schema | LoginPage.jsx, Db/users.sql | TC-01: Verify user can sign in | In Progress |
| **FR-02** | System shall provide a **Login Module Microservice** that will take user's credentials as a Json string and output whether the login was successful or not. Upon successful authentication, the service shall return the user's 2FA authorization and a JWT token. | Functional Requirement | Login validation flow, Login Module Workflow, External Interfaces, Exception Handling | Login.cs, Twilio API Integration, | TC-02: Verify user has been prompted and given a 2FA code | In Progress |
| **FR-03** | System shall provide an **Aggregation Microservice** to receive multiple inputs from external APIs; it shall sue user credentials to scan for breach data. | Functional Requirement | Aggregation Microservice Workflow, External Interfaces, Exception handling | Homepage.jsx, Db/users.sql, function_app.py, risksTrain.py, breach1AI.py | TC-03: Verify microservice processes multiple API inputs | In Progress |
| **FR-04** | System shall provide a **Dashboard Screen User Interface** where logged-in users can access Account Profile, sign up or sign in (if guest), perform quick search, and view user history. | Functional Requirement | Dashboard Screen UI Workflow, External Interfaces, Exception handling | Homepage.jsx, Login.CS, | TC-04: Verify user can access each field on the Dashboard screen | In Progress |
| **FR-05** | System shall provide an **AI Model** that determines risk assessment based on breach data. | Functional Requirement | AI Model Workflow, External Interfaces, Exception handling | function_app.py, risksTrain.py, breach1AI.py | TC-05: Verify AI model determines risk assessment accurately | In Progress |

**Figure 2**

# Functional Requirements - Login Screen (REQ ID: FR-01)

**Login User Interface:**

The Login User Interface is the first screen the user will access when using the web application. Listed below are the system requirements of the Login User Interface screen.

**WBS Tasks: 1.1.2, 1.1.3 3.1.1**

I/O Requirements

The Login Interface will feature two input elements – Username/Email and Password. These are string input fields. There will also be a button element for the user to submit their credentials. These elements will allow the user to enter in characters the represent their username or password.

On successful login, the user will be redirected to the user dashboard. On unsuccessful login, the user will be directed to the error screen, and be unable to get access to the service. The user also has the option to use the service as a guest.

Workflow

User enters credentials

1. Credentials are sent to login microservice when user presses 'Submit' button
2. The 'Submit' button will be unusable unless the input fields are properly filled
3. Depending on the results of the microservice and the 2-FA validation:
4. If successful, the user will be redirected to account Dashboard screen
5. If unsuccessful, the user will be directed to an error screen, and be unable to get access to the service

**Though the Login Screen and Login Module are both seperate Functional Requirements of this project, the workflow diagram on Page 17 combines the workflow, as they both work together in providing login access for JaGuard.**

# Functional Requirements – Login Screen (REQ ID: FR-01)

External Interfaces

- User Interfaces:
  - Text Input Fields – Username/Email and Password
  - Button – Submit Credentials
  - Button – 'Continue as Guest'
  - Button – New Account Registration
- Software Interfaces
  - Screen that is running on a React/Vite
  - Login Microservice Access
  - 2-FA Twilio Service
- Hardware Interfaces
  - JaGuard runs using a web browser.
- Communication Interfaces
  - The JaGuard login screen will communicate directly with the login/logger microservice, and the Twilio 2-FA to verify user authenticity.
  - Communication processes use HTTPS protocols

Exception Handling

- E1.1: Invalid Input
- E1.2: Expired 2FA Code
- E1.3: Microservice Failure

When any exception occurs, including input or internal errors, it will result in an error being returned to the user, prompting them with instructions to re-try, or continue as a guest

# Functional Requirements – Login Module ( REQ ID: FR-02)

**Login Module Microservice:**

I/O Requirements

The Login Module Microservice (ID: 1) will take an input JSON string (formatted from the frontend Login Screen UI inputs) with the following parameters attached:

1. Username/Email
2. Password

The Login Module will output a JSON string with a message indicating whether the login was successful or not, the 2-FA authorization status, and if successful also returning a JWT token.

**WBS Related: 1.1.2, 1.1.3, 1.1.4, 1.2.1, 1.2.2, 1.1.3**

Workflow

1. JSON payload received and serialized into object
2. User password is hashed using SHA-256
3. None of the passwords are stored in plaintext
4. Parameters are entered into a SQL command, and will query the database looking for a user match in the User's Table.
5. If a match is found between the username and hashed password, then the user's account information will be returned back to the microservice.
6. The Login Microservice will use another microservice to trigger a 2-FA prompt to the user using Twilio
7. If the 2-FA is successful, the user's information will be returned to the frontend along with a generated session token to grant user access to the service.

**Though the Login Screen and Login Module are both seperate Functional Requirements of this project, the workflow diagram on page 17 combines the workflow, as they both work together in providing login access for JaGuard.**

# Functional Requirements – Login Module (REQ ID: FR-02)

External Interfaces

- User Interfaces:
  a. The login microservice does not contain a UI, however, the login screen contains the User Interface that communicates with the microservice.
- Software Interfaces
  b. The login microservice uses Azure Functions
  c. Uses tools such as oAuth Verification, JWT token management
  d. Uses MySQL server for database management and queries
- Hardware Interfaces
  e. Uses cloud-based Azure Functions tools provided by Microsoft.
  f. On the local scale, uses Docker Containers use the login microservice along with the DB.
- Communication Interfaces
  g. Twilio Integration
    - Uses Twilio for user 2-FA
  h. MySQL Server
    - Database Management tool storing user data
  i. Login Screen – React Frontend
    - User Interface to input credentials to be sent to the microservice

Exception Handling

- E2.1: Input Validation – Invalid JSON
- E2.2: Failure to Find Account
- E2.3: Failure to Authenticate with 2-FA
- E2.4: Other Exceptions from Internal Errors

Any and all errors will result in the failure being logged, and return an error message back to the frontend, informing the user of the error and prompting them to try again.

# Workflow Diagram – Login Screen and Login Module Functional Requirements(FR-01 & FR-02)

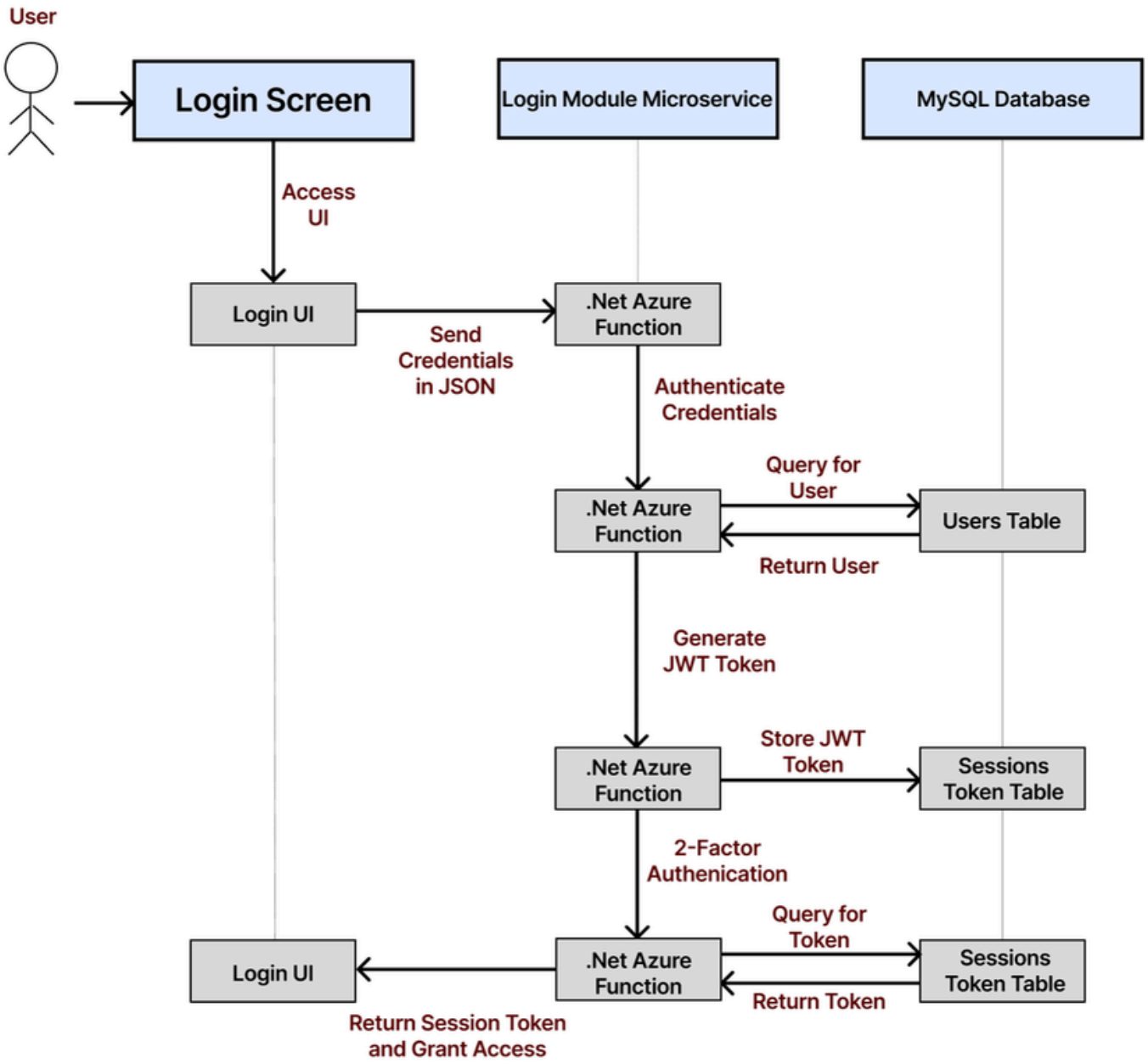



**Figure 3**

# Functional Requirements - Aggregation (FR-03)

**Aggregation Microservice**

I/O Requirements

The Aggregation Microservice will take many different inputs from the following External API's:

1. Have I been Pwned
2. XposedOrNot
3. LeakCheck
4. Leak-Lookup
5. Webz.io Dark Web API

Along with the External API's, the microservice requires a JSON string from the Dashboard Search, containing at least on of the following credentials to be scanned:

- Username
- Email
- Password

**WBS Related:** 2.1.1, 2.1.2, 2.1.3

Workflow

1. The scan service from the UI frontend will trigger this microservice, using the provided username, email, and/or password and processing them into a data object.
2. This data is deleted immediately after use, and the password used for data breach searching IS NOT SAVED in any way. The only results that are saved is the results of the breach, removing all PII.
3. This data object is then formatted into JSON strings and then used to call the external APIs to collect data breach information on them.
4. All the data provided from the external API's are then formatted into a summarized result, and an AI model will process the data to perform Risk Assessments to give details on the provided breaches and user feedback.
5. If the User has an account and is logged in, the breach data (just containing the Username and/or email, no password) will be logged and stored in the DB.
6. This summarized result and AI assessment result will be returned to the frontend User Interface in a JSON string format.

# Functional Requirements - Aggregation (FR-03)

External Interfaces

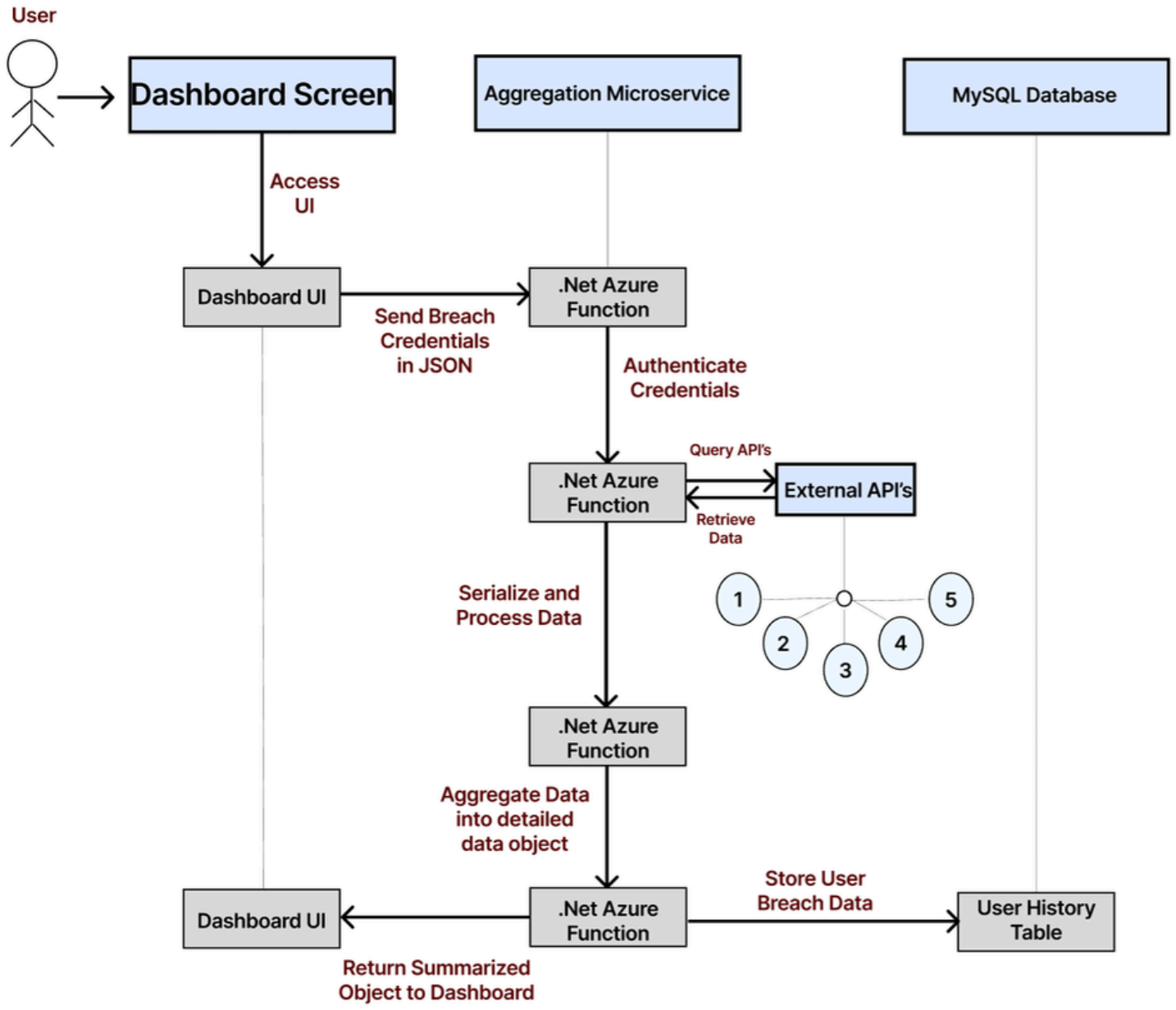
- User Interfaces:
  a. The aggregation microservice does not contain a UI, however, the Dashboard search UI will directly call this microservice
  b. Software Interfaces
  c. The aggregation microservice uses Azure Functions
- Hardware Interfaces
  d. Uses cloud-based Azure Functions tools provided by Microsoft.
  e. On the local scale, uses Docker Containers use the login microservice along with the DB.
- Communication Interfaces
  f. Python AI-Model Azure Function
  g. Dashboard User-Interface
  h. MySQL Server DB

Exception Handling

- E3.1: Input Validation – Invalid JSON
- E3.2: External API's Failure
  - In this event, all successful API calls will be aggregated into a final result. Any unsuccessful API calls will be logged, and the user will be informed that some external tools failed, but will be given the results of successful call.
- E3.3: Internal Microservice Error
- E3.4: Other Exceptions from Internal Errors

All errors will result in the failure being logged, and may return an error message to the UI depending on the error.

# Workflow Diagram – Aggregation Microservice (FR-03)




Number Key:
1. LeakCheck
2. Have I Been Pwned
3. XposedOrNot
4. Leak-Lookup
5. Webz.IO Dark Web API

**Figure 4**

# Functional Requirements - Dashboard (FR-04)

**Dashboard Screen User Interface**

I/O Requirements

The Dashboard screen has many different UI input fields.

1. Account Profile Button (if-logged in)
2. Sign-Up/Login (if Guest)
3. Quick Search Input
4. User History

Depending on whether what input field is used, different application services/features are utilized in the output.

**WBS Related: 3.1.2, 3.1.5**

Workflow

Each individual Input option on the Dashboard screen will result in different workflows and output results. The main workflow to note is the Quick Search Input.

1. Account Profile – Loads Account Profile Screen
2. Sign-Up/Login – Loads Login Screen
3. Quick Search Input
    a. User can specify whether they are entering an email/username/or password
4. User will enter the credentials into the input field
    a. Submitting these credentials will call the aggregation microservice
        i. If successful, the Data Results screen will be loaded.
        ii. If unsuccessful, an error message will be returned

# Functional Requirements - Dashboard (FR-04)

External Interfaces

- User Interfaces:
  a. Button – Account Profile
  b. Button – Sign Up/Login Button
  c. Text Input – Quick Data Breach Search
  d. History Display – Displays past account breach history
- Software Interfaces
  e. JaGuard dashboard is a web-based application
- Hardware Interfaces
  f. Uses React/Vite tools to host web-application, including Dashboard screen
- Communication Interfaces
  g. External Aggregation Microservice
  h. Login UI Screen
  i. Profile Screen
  j. Data Screen

Exception Handling

- E4.1: Invalid Input
- E4.2: Screen loading Failures
- E4.3: Internal Microservice Error
- E4.4: Other Exceptions from Internal Errors

All errors will result in the failure being logged, and may return an error message to the UI depending on the error.

# Dashboard Use Case Diagram (FR-04)

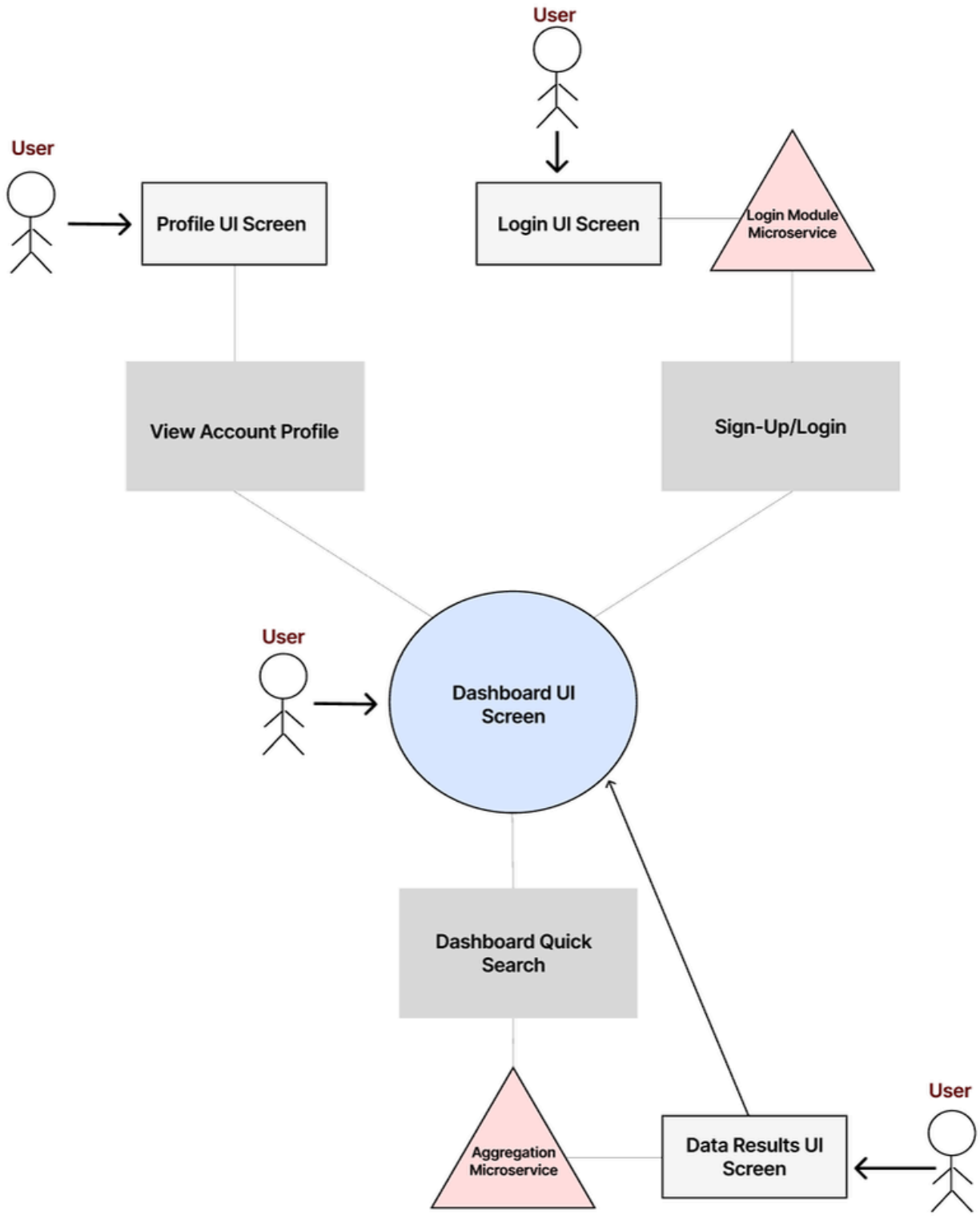
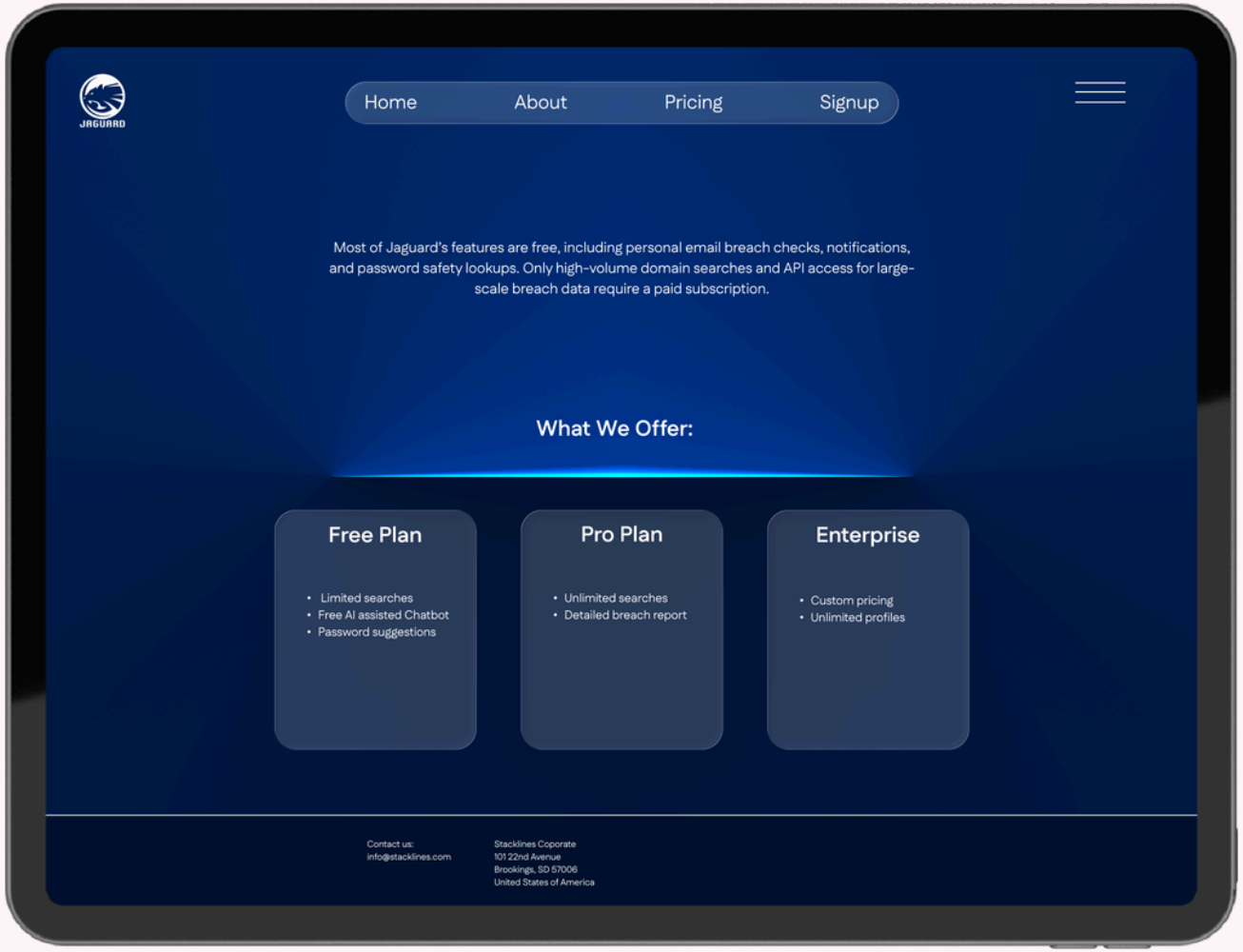



**Figure 5**

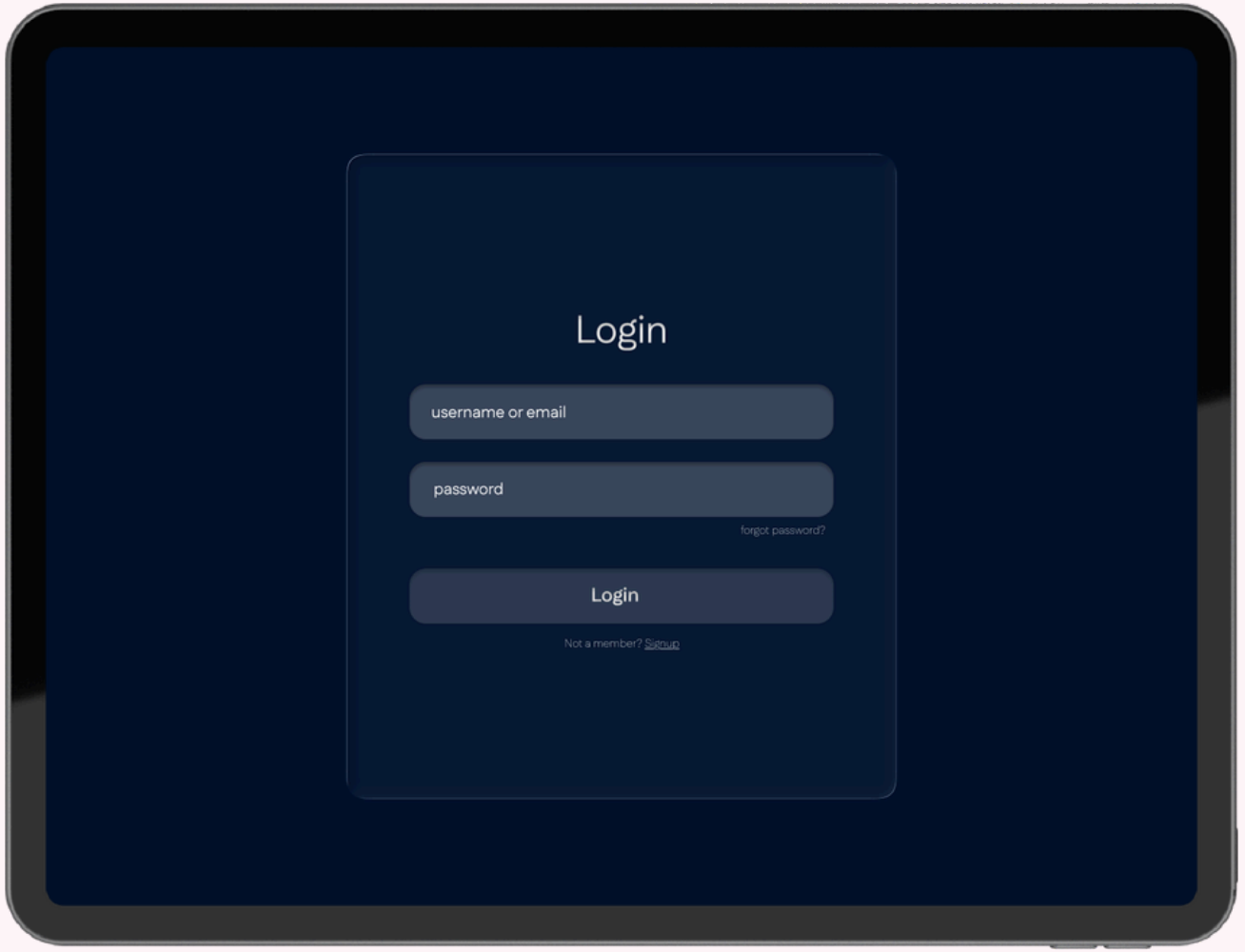
# Dashboard – UI Mockup (FR-04)


**Mockup 1.1**


JAGUARD
Home
About
Pricing
Signup
Most of Jaguard's features are free, including personal email breach checks, notifications, and password safety lookups. Only high-volume domain searches and API access for large-scale breach data require a paid subscription.
What We Offer:
Free Plan
• Limited searches
• Free AI assisted Chatbot
• Password suggestions
Pro Plan
• Unlimited searches
• Detailed breach report
Enterprise
• Custom pricing
• Unlimited profiles
Contact us:
info@stacklines.com
Stacklines Coporate
101 22nd Avenue
Brookings, SD 57006
United States of America

**Mockup 1.2**

# Login Screen – UI Mockup (FR-01)




**Mockup 2.1**


JAGUARD
Home
About
Pricing
Signup
John Doe
History
Breach Analysis
Emails
Passwords
Settings
Upgrade to Pro
BREACH STATS
TOTAL BREACHES
1,875
MONTHLY BREACHES
600
400
200
JAN
FEB
MAR
APR
BREACHES OVER TIME
600
400
200
JAN
FEB
MAR
APR
APR
600
400
200
JAN
2020
2021
2022
APR
APR

**Mockup 2.2**

# Functional Requirements – AI Model (FR-05)

**AI Model**

I/O Requirements

The AI Model requires a JSON input in string format. The AI model will then assess the information using built in business logic to determine a Risk Assessment based on the breach data

**WBS Tasks: 6.1, 6.2, 6.3**

Workflow

1. Model receives JSON string data of breach logs
2. Based on certain patterns in the breach logs, and severity of data breach, the AI model will assess a Risk Score, along with suggesestions the user can follow to secure the account
3. The results the AI generates will be serialized into a data object.
4. This data object will be returned to the frontend results screen, along with the data from the aggregation microservice.

External Interfaces

- User Interfaces:
  a. The AI model has no user interfaces
- Software Interfaces
  b. The AI model is built using PyTorch and other Python AI libraries and tools
- Hardware Interfaces
  c. Uses a Docker Container to host Python Azure Functions to access this AI model
- Communication Interfaces
  d. External Aggregation Microservice
  e. Data Results Screen

Exception Handling

- E5.1: Invalid JSON input
- E5.2: AI Model Failure
- E5.3: Other Exceptions

Depending on the error, the AI model may not be able to properly make an assessment. In such a case, the error will be logged, and the Risk Assessment and Feedback may be excluded from the data results screen.

# AI Model Workflow Diagram (FR-05)

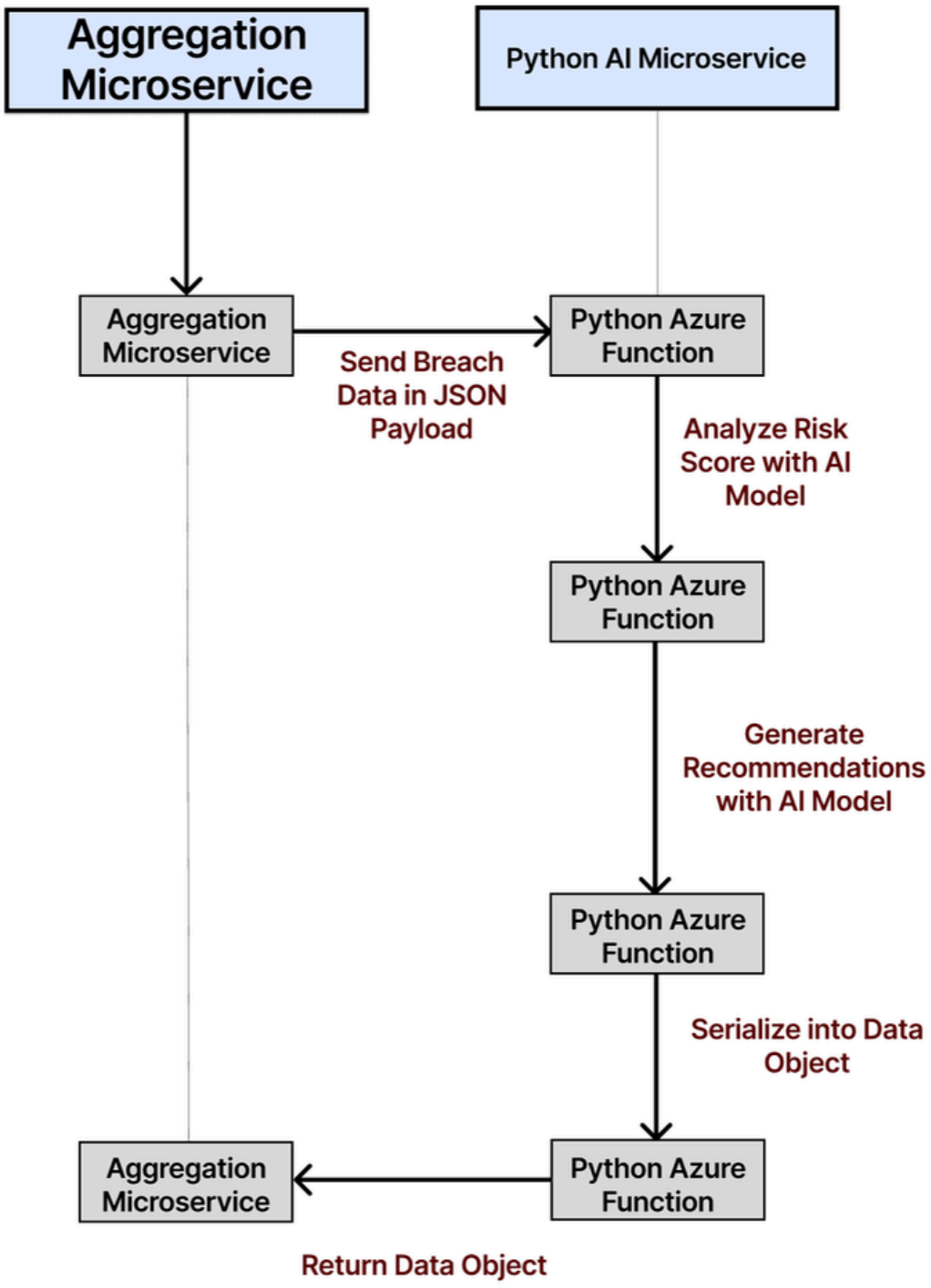



**Figure 6**

# Traceability Matrix Nonfunctional Requirements

**Legend**

| Color | Meaning |
|---|---|
| | Completed |
| | In Progress |
| | Not Started |

| Req ID | Requirement Description | Type | Design Component | Implementation Module | Test Case | Status |
|---|---|---|---|---|---|---|
| NFR-01.1 | The system shall support at least 20 concurrent users and will not exceed an average response time of 3 seconds per request under normal conditions. | Nonfunctional Requirement | System architecture diagram | Docker-compose.yml, function_app.py | | In Progress |
| NFR-01.2 | All web pages shall load within 3 seconds over 49 Mbps of internet speed on modern web browsers (Chrome version 141+, Firefox version 126+, Edge 141+). | Nonfunctional Requirement | UI design, Optimization workflow | LoginPage.jsx, homepage.jsx | | In Progress |
| NFR-01.3 | The system shall generate logs and create user and admin reports within 10 seconds under normal server conditions. | Nonfunctional Requirement | Logging and reporting workflow diagram | Docker-compose.yml, function.py | | In Progress |
| NFR-02.1 | The system will be deployed using Azure Functions for backend processing and shall be executed successfully on local machines. | Nonfunctional Requirement | Deployment architecture diagram | Function_app.py, Docker-compose.yml | | In Progress |
| NFR-02.2 | The system shall be accessible using modern web browsers (Chrome version 141+, Firefox version 126+, Edge 141+) | Nonfunctional Requirement | UI compatibility design | LoginPage.jsx, homepage.jsx | | In Progress |
| NFR-02.3 | The system will be compatible with the hosting environment's updates, such as React v19. Compatibility will be verified after each environmental update that occurs every 3 months. | Nonfunctional Requirement | Environment compatibility plan | Docker-compose.yml | | In Progress |
| NFR-03.1 | The system shall maintain a minimum of 99.9% uptime between 6:00 AM and 10:00 PM CT; this will be measured over a monthly operational period. | Nonfunctional Requirement | Monitoring & deployment design | Docker-compose.yml | | In Progress |
| NFR-03.2 | 2FA service shall successfully respond to 99% of authentication requests within 5 seconds during high-demand periods of 20 concurrent login sessions. | Nonfunctional Requirement | Authentication workflow diagram | Login.cs, Twilio API integration | | In Progress |
| NFR-03.3 | The System shall automatically restart all services within 1 minute of an unexpected crash or freeze; this will be verified by the monitoring of logs. | Nonfunctional Requirement | System recovery & Exception handling flow | Docker-compose.yml, function_app.py | | In Progress |

**Figure 7.1**

# Traceability Matrix Nonfunctional Requirements (Cont)

**Legend**

| Color | Meaning |
|---|---|
| | Completed |
| | In Progress |
| | Not Started |

| Req ID | Requirement Description | Type | Design Component | Implementation Module | Test Case | Status |
|---|---|---|---|---|---|---|
| NFR-04.1 | All user passwords and sensitive data shall be hashed using SHA-256 for encryption before storage in the database. | Nonfunctional Requirement | Security architecture diagram | Login.cs, Db/users.sql | | In Progress |
| NFR-04.2 | The system shall perform automatic daily backups of all user and application data; they will be stored in an encrypted location and validated weekly for data integrity. | Nonfunctional Requirement | Backup process design | Docker-compose.yml | | In Progress |
| NFR-04.3 | The system shall sanitize database inputs to prevent SQL injection vulnerabilities; this will be verified by using the penetration test tools. | Nonfunctional Requirement | Input validation & Security design | Login.cs, function_app.py | | In Progress |
| NFR-05.1 | System architecture shall use modular components, allowing features to be added or modified without impacting current functionality. This will be verified by testing all code changes to make sure they have not negatively impacted any existing code or functionality. | Nonfunctional Requirement | Modular architecture | Docker-compose.yml, Login.cs, function_app.py | | In Progress |
| NFR-05.2 | Documentation shall be reviewed and updated with every major release within 48 hours of deployment. | Nonfunctional Requirement | Documentation Maintenance plan | N/A | | In Progress |
| NFR-05.3 | Planned maintenance windows shall be announced no later than 48 hours in advance, and notification logs shall confirm delivery to affected user groups. | Nonfunctional Requirement | Maintenance Scheduling Workflow | N/A | | In Progress |
| NFR-06.1 | The user interface shall use consistent and clearly labeled buttons and text boxes so that 85% of test users can log in or have a user credential checked within 2 minutes without assistance. | Nonfunctional Requirement | UI design guidelines | LoginPage.jsx, homepage.jsx | | In Progress |
| NFR-06.2 | Navigation and visual design shall remain consistent across all pages; this will be verified by examining all UIs of our web pages to ensure 100% consistency. | Nonfunctional Requirement | UI style guide & Navigation design | LoginPage.jsx, homepage.jsx | | In Progress |

**Figure 7.2**

# Traceability Matrix Nonfunctional Requirements (Cont)

**Legend**

| Color | Meaning |
|---|---|
| | Completed |
| | In Progress |
| | Not Started |

| Req ID | Requirement Description | Type | Design Component | Implementation Module | Test Case | Status |
|---|---|---|---|---|---|---|
| NFR-07.1 | At least two training sessions shall be delivered to all administrators and power users prior to production release, with attendance records. | Nonfunctional Requirement | Training & onboarding plan | N/A | | In Progress |
| NFR-07.2 | An AI-based chatbot feature shall be accessible from all pages and respond to user questions within 7 seconds. | Nonfunctional Requirement | Help system architecture diagram | Homepage.jsx, function_appy.py | | In Progress |
| NFR-08.1 | The system shall display clear and non-technical error messages for all users who are facing exceptions; this will be verified by checking out the UI with simulated error messages. | Nonfunctional Requirement | Exception handling workflow, UI error modal design | LoginPage.jsx, Login.cs | | In Progress |
| NFR-08.2 | All critical errors will be logged with user ID, error description, and timestamp; these will be reviewed by admin users to fix system issues. | Nonfunctional Requirement | Logging architecture design | Function_app.py, docker-compose.yml | | In Progress |
| NFR-08.3 | The system shall send a critical system failure alert within 1 minute of detection to administrators. | Nonfunctional Requirement | Alert & Monitoring design | docker-compose.yml | | In Progress |
| NFR-09.1 | Each critical feature shall undergo individual testing to verify functionality as well as the inputs and outputs. | Nonfunctional Requirement | Unit Test Plan | Login.cs, homepage.jsx, LoginPage.jsx | | In Progress |
| NFR-09.2 | Integration testing shall confirm that all modules can interact with each other without unhandled exceptions or data loss. | Nonfunctional Requirement | Integration test workflow | docker-compose.yml, | | In Progress |
| NFR-09.3 | System performance degradation shall not exceed 15% when load and stress testing for 20 concurrent users. | Nonfunctional Requirement | Performance & Load testing plan | Docker-compose.yml, function_appy.py | | In Progress |

**Figure 7.3**

# Non-Functional Requirements

1. **Performance Requirements (NFR-01)**

- **NFR-01.1:** The system shall support at least 20 concurrent users and will not exceed an average response time of 3 seconds per request under normal conditions.
- **NFR-01.2:** All web pages shall load within 3 seconds over 49 Mbps of internet speed on modern web browsers (Chrome version 141+, Firefox version 126+, Edge 141+).
- **NFR-01.3:** The system shall generate logs and create user and admin reports within 10 seconds under normal server conditions.

2. **Operational and Environment Requirements (NFR-02)**

- **NFR-02.1:** The system will be deployed using Azure Functions for backend processing and shall be executed successfully on local machines.
- **NFR-02.2:** The system shall be accessible using modern web browsers (Chrome version 141+, Firefox version 126+, Edge 141+)
- **NFR-02.3:** The system will be compatible with the hosting environment's updates, such as React v19. Compatibility will be verified after each environmental update that occurs every 3 months.

3. **Reliability and Availability (NFR-03)**

- **NFR-03.1:** The system shall maintain a minimum of 99.9% uptime between 6:00 AM and 10:00 PM CT; this will be measured over a monthly operational period.
- **NFR-03.2:** 2FA service shall successfully respond to 99% of authentication requests within 5 seconds during high-demand periods of 20 concurrent login sessions.

# Non-Functional Requirements (Cont)

3. **Reliability and Availability (NFR-03) (Cont)**

- **NFR-03.3:** The System shall automatically restart all services within 1 minute of an unexpected crash or freeze; this will be verified by the monitoring of logs.

4. **Backup and Security (NFR-04)**

- **NFR-04.1:** All user passwords and sensitive data shall be hashed using SHA-256 for encryption before storage in the database.
- **NFR-04.2:** The system shall perform automatic daily backups of all user and application data; they will be stored in an encrypted location and validated weekly for data integrity.
- **NFR-04.3:** The system shall sanitize database inputs to prevent SQL injection vulnerabilities; this will be verified by using the penetration test tools.

5. **Maintainability (NFR-05)**

- **NFR-05.1:** System architecture shall use modular components, allowing features to be added or modified without impacting current functionality. This will be verified by testing all code changes to make sure they have not negatively impacted any existing code or functionality.
- **NFR-05.2:** Documentation shall be reviewed and updated with every major release within 48 hours of deployment.
- **NFR-05.3:** Planned maintenance windows shall be announced no later than 48 hours in advance, and notification logs shall confirm delivery to affected user groups.

# Non-Functional Requirements (Cont)

6. **Usability (NFR-06)**
   - **NFR-06.1:** The user interface shall use consistent and clearly labeled buttons and text boxes so that 85% of test users can log in or have a user credential checked within 2 minutes without assistance.
   - **NFR-06.2:** Navigation and visual design shall remain consistent across all pages; this will be verified by examining all UIs of our web pages to ensure 100% consistency.
7. **Documentation and Training (NFR-07)**
   - **NFR-07.1:** At least two training sessions shall be delivered to all administrators and power users prior to production release, with attendance records.
   - **NFR-07.2:** An AI-based chatbot feature shall be accessible from all pages and respond to user questions within 7 seconds.
8. **Exception Handling (NFR-08)**
   - **NFR-08.1:** The system shall display clear and non-technical error messages for all users who are facing exceptions; this will be verified by checking out the UI with simulated error messages.
   - **NFR-08.2:** All critical errors will be logged with user ID, error description, and timestamp; these will be reviewed by admin users to fix system issues.
   - **NFR-8.3:** The system shall send a critical system failure alert within 1 minute of detection to administrators.

# Non-Functional Requirements (Cont)

9. **Testing Requirements (NFR-09)**

- **NFR-09.1:** Each critical feature shall undergo individual testing to verify functionality as well as the inputs and outputs.
- **NFR-09.2:** Integration testing shall confirm that all modules can interact with each other without unhandled exceptions or data loss.
- **NFR-09.3:** System performance degradation shall not exceed 15% when load and stress testing for 20 concurrent users.

# Change Control

1. **Change Control Outline**
   - Major changes to existing code or any documentation that has been finalized should follow the guidelines listed in this document. When changing documentation or code the team member that brought up the changes needs to submit a change request (CR) to the project manager. Once the project manager looks at the CR they should meet with the team members that will be affected by the changes and weigh the impact of the project's scope, schedule, resources, and budget before approving the CR.
2. **Change Control Log**
   - Any change request should follow a template so the changes can be put into a table and are easily identifiable. Change control logs are different from Git issues, which are for bug fixes and small tweaks. The change log should include:
   - Change ID: A unique number to identify the request (e.g. CR-001).
   - Date submitted.
   - Requested by: The name and role of the person who suggested the change.
   - Description of change: A detailed description of the change suggested.
   - Reason for change: A detailed description of why the change was needed.
   - Impact analysis: A rough prediction of the impact on other parts of the project (e.g. time, cost, conflicting systems, and scope).
   - Decision: The final decision will either be approved, rejectors, or deferred.
   - Date of decision: The date the decision was made.

# Change Control Workflow Diagram

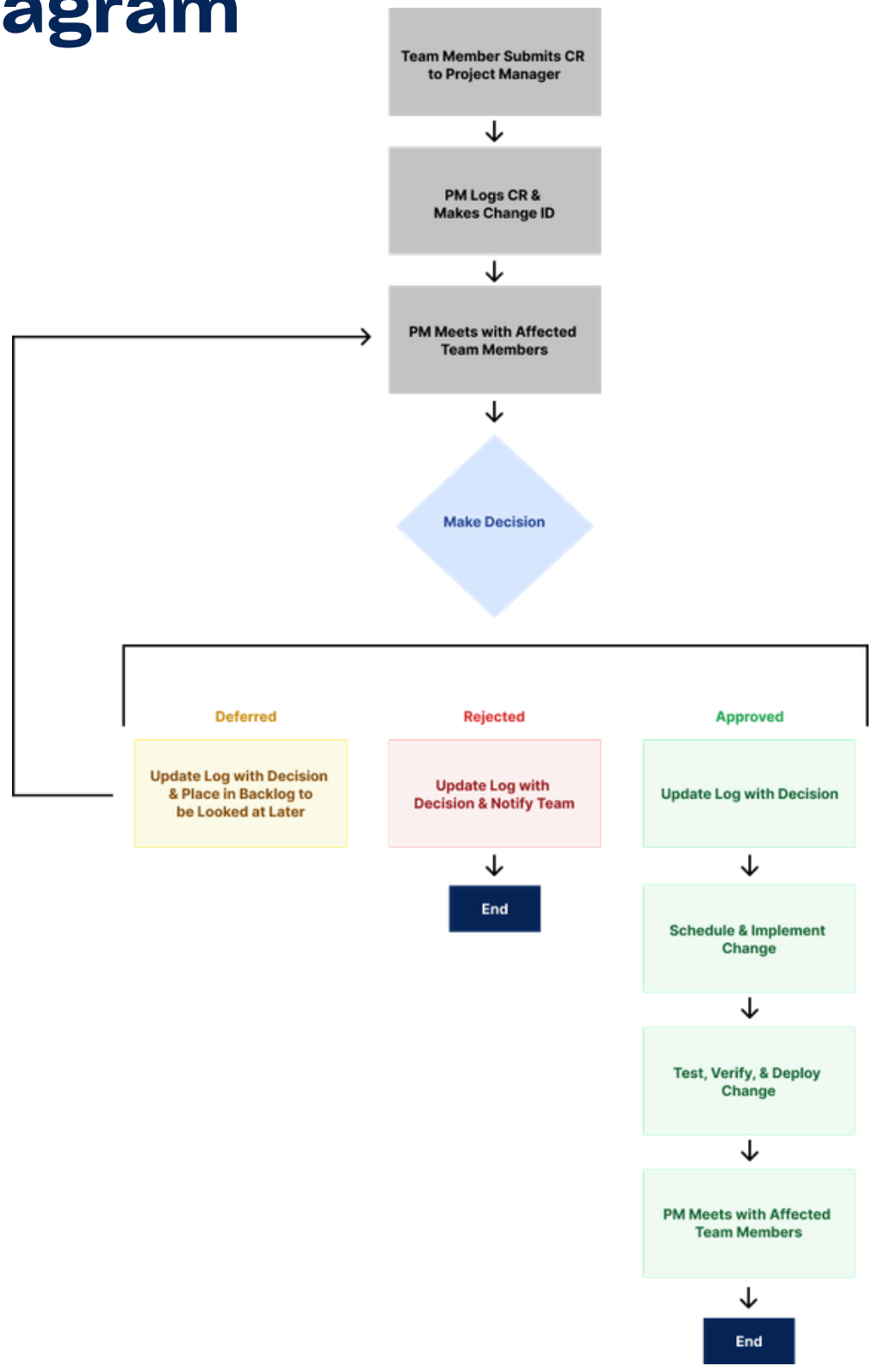



**Figure 8**

# Appendix (Definition/Acronyms/Abbreviations, Signature part of Client)

The appendix includes additional information for Requirements Document and definitions, acronyms and abbreviations. It also includes the signature of the client.

- 2-FA: Two Factor Authentication
- API: Application Programming Interface
- Auth: Authentication
- DB: Database
- ER: Entity Relationship Dlagram
- JWT: Json Web TOken
- Jaguard: Name of the system - cloud based data breach system
- RD: Requirements Documentation
- UI: User Interface
- UML - Unified Modeling Language
- WBS: WOrk Breakdown Structure
- CR: Change request

**Signature of the client**

This section is for the client's approval and signature.

| Name of the Client | Position | Signature | Date |
|---|---|---|---|
| Chulwoo Pack | Assistant Professor | _____ | _____ |

# Work Breakdown Structure

The Work Breakdown Structure has been given some minor adjustments, with the Introduction of an AI Header and associated tasks. These additions allow for a more realistic time-scale for AI-Model development and integration into the JaGuard application.

on of new AI-Focused Tasks, denoted by 6.

**New Tasks Introduced**

6.1 – Develop and Train AI Model

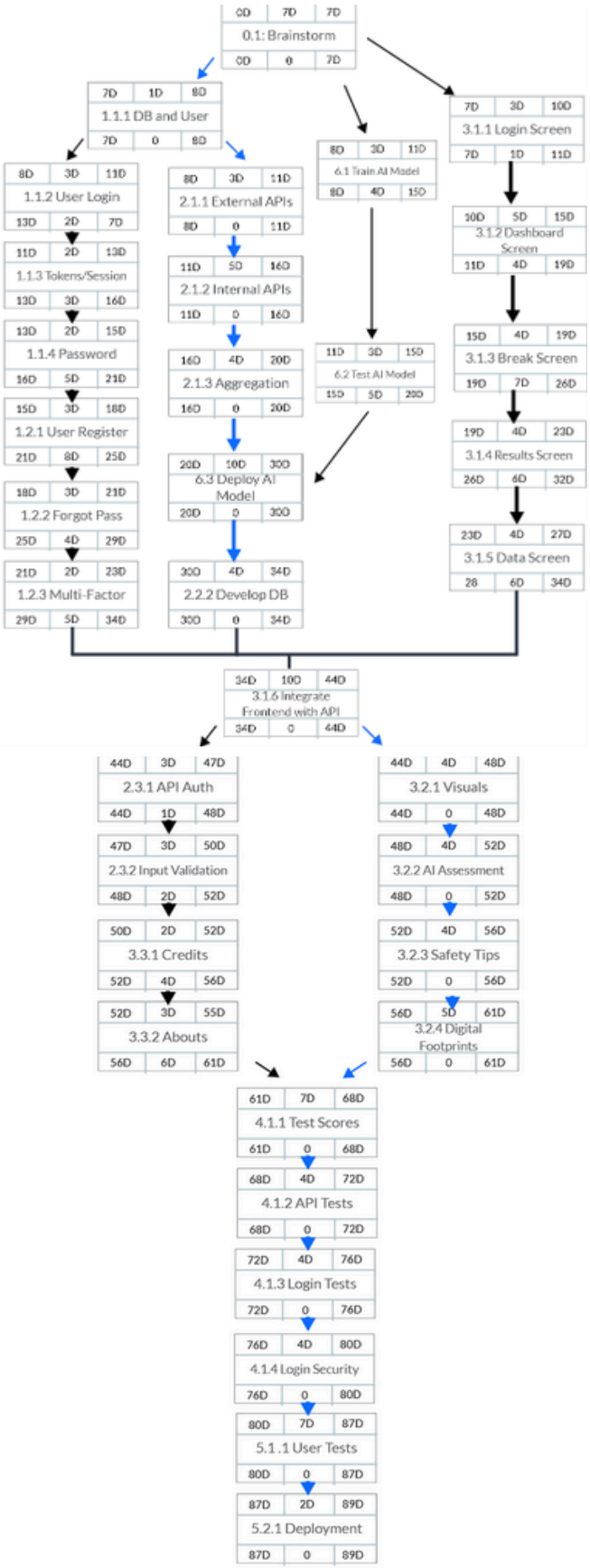
6.2 – Test, Re-Factor, and Retrain AI Model

6.3 – Deploy AI Model and Integrate Azure Function Usage

**The Work Breakdown Structure Diagram is on the following page. Refer to Figure 2.**

## Work Breakdown Structure

### Figure 9



# UML Diagram

The Application Layer handles authentication and token management, while the Business Layer  is responsible with identifying and reporting data breaches. The Database Layer persists user and log data securely, and the External Services layer integrates with third-party APIs to fetch real-time breach data. **(Figure 10)**

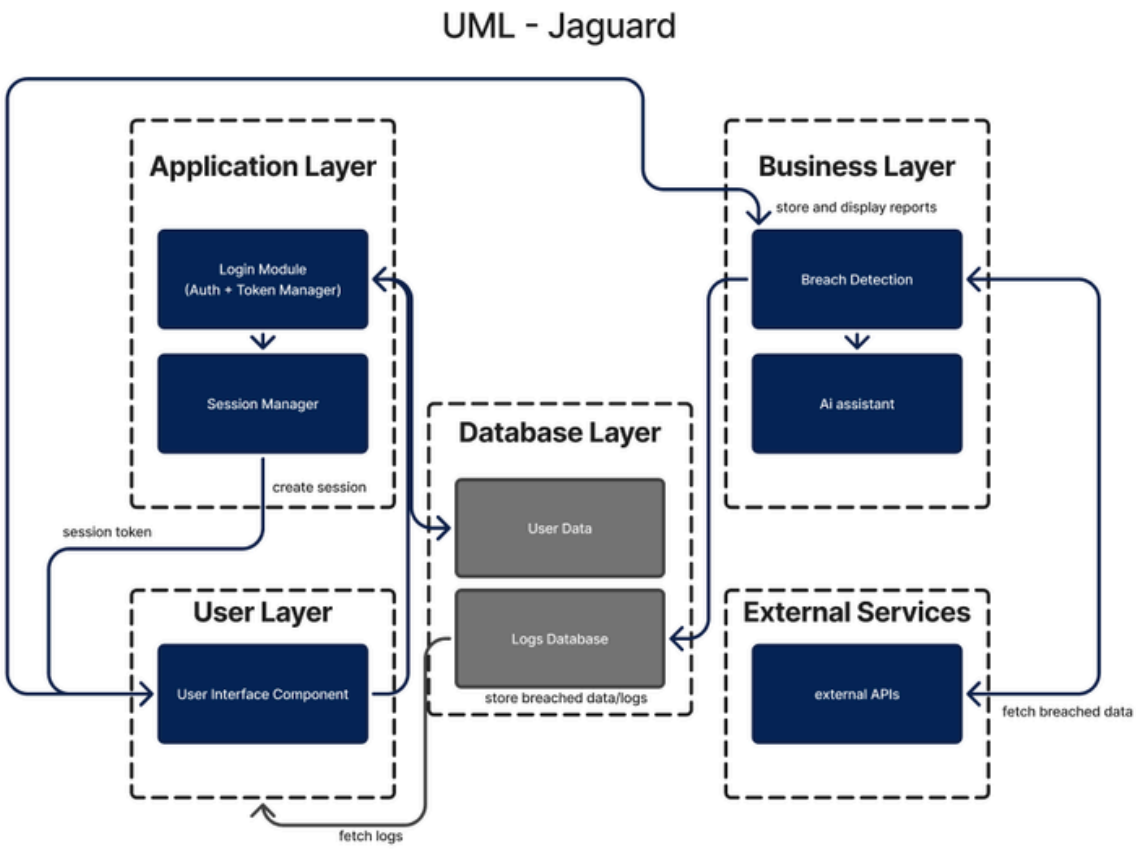



**Figure 10**

# UML (continued)

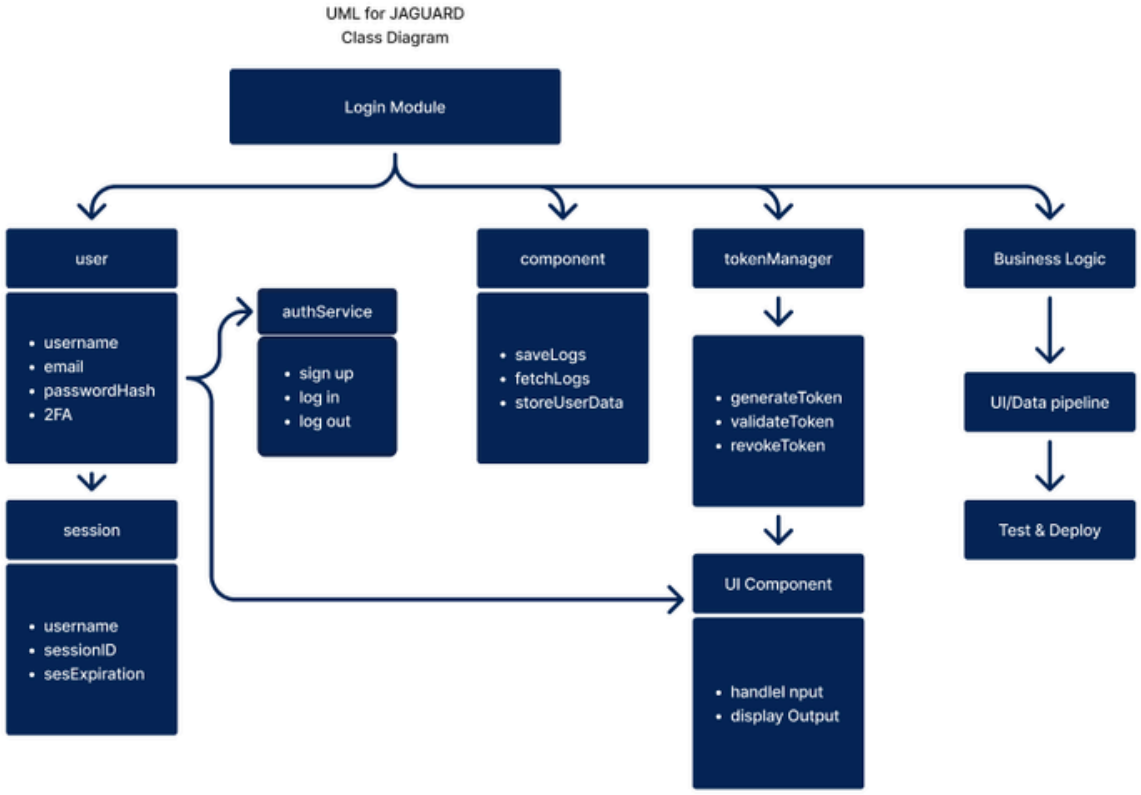



**Figure 11**

The system's structural design, or class diagram, is illustrated in figure 4. While the Session class handles session parameters like session ID and expiration, the User class contains important user data including username, email, hashed password, and 2FA settings. The authService class serves as the entry point for the authentication procedure and offers methods for user signup, login, and logout. Session token creation, validation, and revocation are managed by the TokenManager. The Component class handles data-related functions, such as storing user data, retrieving logs, and saving logs. User input handling and output display fall under the control of the UI Component class.

# Gantt Chart

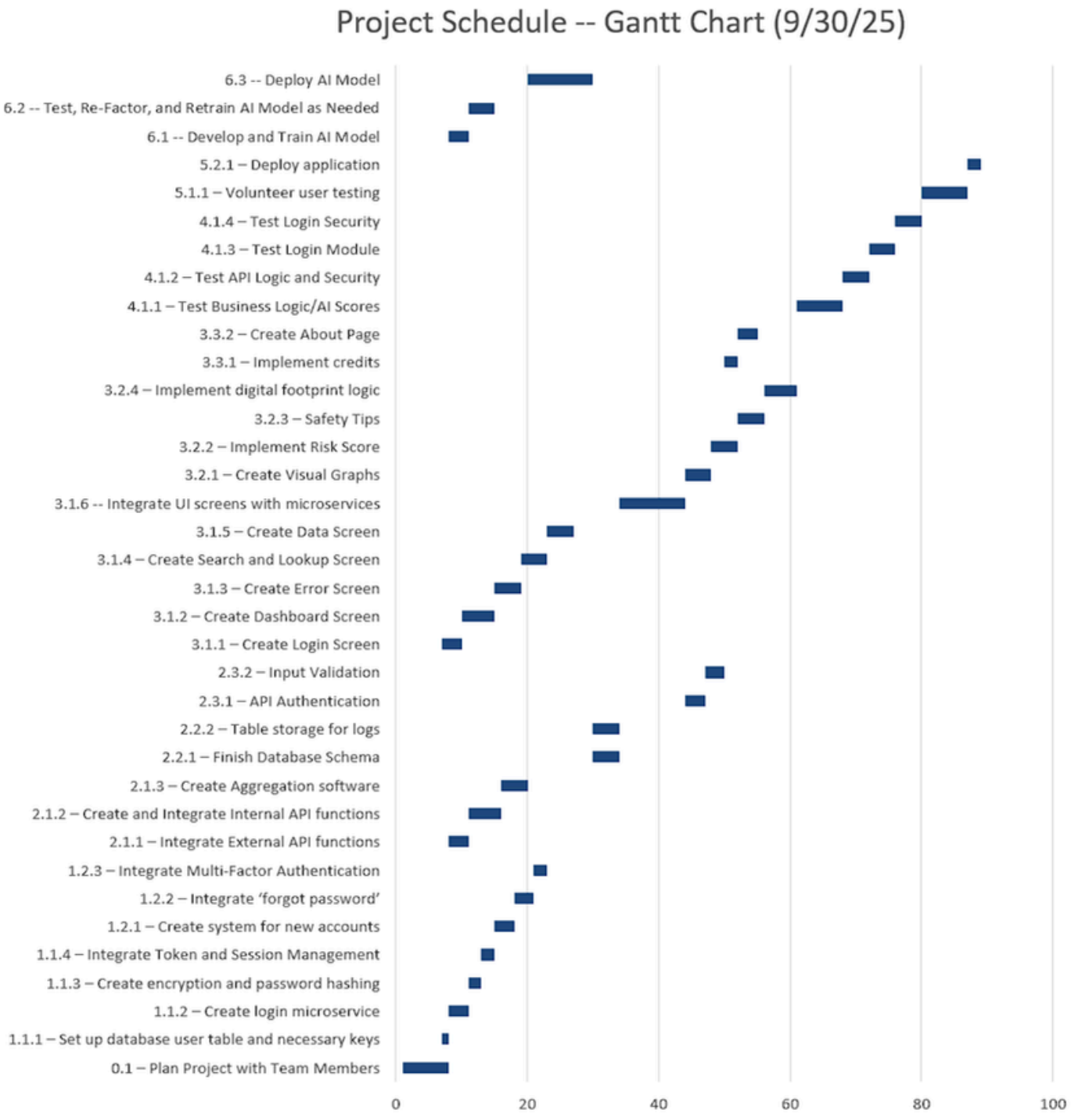



**Figure 12**

# ER Diagram

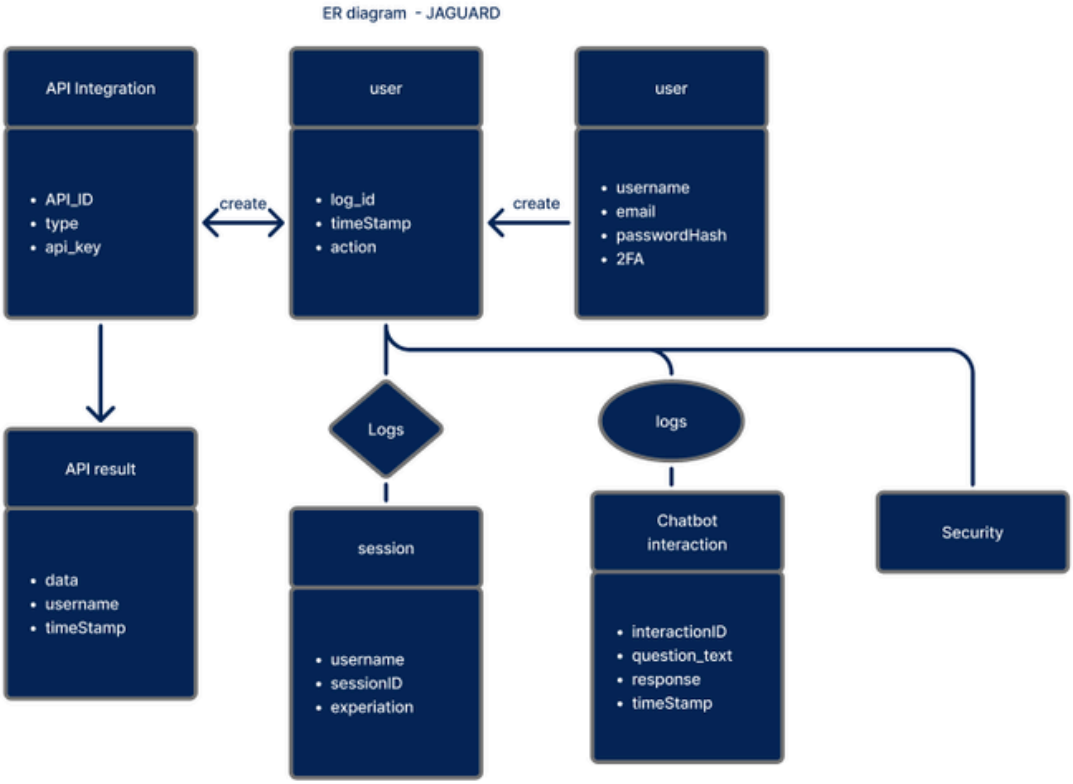



**Figure 13**

The Jaguard ER diagram illustrates the relationship between users, sessions, APIs, and chatbots in a cloud-based breach monitoring system. Sessions control authentication and security, while logs record user actions, API requests, and chatbot inquiries. API results store compromised data, and the security component implements protection measures across the system. The figure as a whole shows how Jaguard tracks and secures actions.

# Terms and Conditions

**Access**

Only authorized team members and instructors may use or test the system. Login credentials should not be shared.

**Service Agreement**

A service agreement will be provided between parties for expectaions and performance of the project along with a time frame of completion.

**Data**

Any data used in this system will be handled carefully and stored securely. Sensitive information such as passwords will be protected or deleted when no longer needed.

**Confidentiality**

Any information shared during the course of the project will be kept confidential and will only be used for activities related to the project.

**Warranty**

System will be delivered “as is”. For 60 days after project completion, our team will continue to provide support and address any issues at no additional cost. Beyond this period, further assistance will be available under a paid support agreement.

**Support**

Support and updates are limited to the duration of the course and project timeline. Along with the 60 days after delivery

Limitations

The system is not intended for commercial or real-world deployment outside of this project.

**Termination**

At any point in time either party can terminate the agreement if any project terms have been violated by either party and or if certain circumstances have come up and the project can longer move forward.

# References (IEEE Format)

GeeksforGeeks, “Unified Modeling Language (UML) Diagrams,” GeeksforGeeks, Oct. 27, 2017. https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml introduction/

GeeksforGeeks, “Introduction of ER Model,” GeeksforGeeks, Jul. 23, 2025. https://www.geeksforgeeks.org/dbms/introduction-of-er-model/

OpenAI, “ChatGPT,” ChatGPT, 2025. https://chatgpt.com/